

Arzel Andrei Israel
Lynxie Jagonob
John Walter Pediangco
John Christian Setubal
Emmanuel Siasstres

Lab 7: Flask Database Integration (The ORM Abstraction)

Side Info: Object-Relational Mapper, is a programming technique used to convert data between incompatible type systems: Object-Oriented Programming (OOP) languages (like Python, Java, or JavaScript) and Relational Databases (like PostgreSQL, MySQL, or SQL Server). Essentially, it allows you to interact with your database using the language you are already writing in, rather than writing raw SQL.

Objective: Build a persistent "Book Collection" API.

Setup Instructions:

1. Install Flask-SQLAlchemy:

o Command: `pip install flask-sqlalchemy`

```
PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pediangco_Setubal_Siasstres_lab_7\flask-db-lab> pip install flask-sqlalchemy
Collecting flask-sqlalchemy
  Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl.metadata (3.4 kB)
Requirement already satisfied: flask>=2.2.5 in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask-sqlalchemy) (3.1.3)
Collecting sqlalchemy>=2.0.16 (from flask-sqlalchemy)
  Downloading sqlalchemy-2.0.51-cp313-cp313-win_amd64.whl.metadata (9.8 kB)
Requirement already satisfied: blinker>=1.9.0 in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (1.9.0)
Requirement already satisfied: click>=8.1.3 in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (8.4.1)
Requirement already satisfied: itsdangerous>=2.2.0 in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask>=2.2.5->flask-sqlalchemy) (3.1.8)
Requirement already satisfied: colorama in C:\Users\emman\AppData\Local\Programs\Python\Python313\Lib\site-packages (from click>=8.1.3->flask>=2.2.5->flask-sqlalchemy) (0.4.6)
Collecting greenlet>=1 (from sqlalchemy>=2.0.16->flask-sqlalchemy)
  Downloading greenlet-3.5.1-cp313-cp313-win_amd64.whl.metadata (3.9 kB)
Collecting typing_extensions>=4.6.0 (from sqlalchemy>=2.0.16->flask-sqlalchemy)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl (25 kB)
Downloading sqlalchemy-2.0.51-cp313-cp313-win_amd64.whl (2.1 MB)
   2.1/2.1 MB 1.0 MB/s 0:00:02
Downloading greenlet-3.5.1-cp313-cp313-win_amd64.whl (238 kB)
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Installing collected packages: typing_extensions, greenlet, sqlalchemy, flask-sqlalchemy
Successfully installed flask-sqlalchemy-3.1.1 greenlet-3.5.1 sqlalchemy-2.0.51 typing_extensions-4.15.0
PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pediangco_Setubal_Siasstres_lab_7\flask-db-lab>
```

2. Project Structure: Create a file named `book_api.py`.

```
FLASK... book_api.py
1 Generate code (Ctrl+I), or select a language (Ctrl+K M). Start typing to dismiss or don't show this again.
```

Hints:

Follow these logical steps to build your persistent Python API.

Step 1: Configuration & Connection

- The "Interface": Import Flask and SQLAlchemy.
- The Config: You must tell Flask where your database file lives.
 - Hint: Set `app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///books.db'`.
- The Instance: Initialize the database by passing your app into the SQLAlchemy class: `db = SQLAlchemy(app)`.

```
1  from flask import Flask, request, jsonify
2  from flask_sqlalchemy import SQLAlchemy
3
4  app = Flask(__name__)
5
6  app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///books.db'
7  app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
8
9  db = SQLAlchemy(app)
```

Step 2: The Model (Defining the Table)

Instead of running a CREATE TABLE script, you define a Class.

- Create a class `Book(db.Model)`.
- Columns: Define an id (Primary Key), a title (String), and an author (String).
- Use `db.Column` and types like `db.Integer` and `db.String`.

```
10
11  class Book(db.Model):
12      id = db.Column(db.Integer, primary_key=True)
13      title = db.Column(db.String(200), nullable=False)
14      author = db.Column(db.String(200), nullable=False)
15
16  with app.app_context():
17      db.create_all()
```

Step 3: Initializing the Database

- You need to tell Flask to actually create the `books.db` file based on your class.
- Use a "shell context" or a simple `with app.app_context(): db.create_all()` block at the start of your script.

```
16  with app.app_context():
17      db.create_all()
```

Step 4: The CRUD Logic

Write your routes using these ORM abstractions instead of raw SQL:

- To CREATE (POST): Create a new instance of your class: `new_book = Book(title="...", author="...")`. Then use `db.session.add(new_book)` and `db.session.commit()`.

```
19 @app.route('/api/books', methods=['POST'])
20 def add_book():
21     data = request.get_json()
22
23     new_book = Book(
24         title=data['title'],
25         author=data['author']
26     )
27
28     db.session.add(new_book)
29     db.session.commit()
30
31     return jsonify({
32         "message": "Book added successfully"
33     })
34
```

- To READ (GET): Use the query interface: `all_books = Book.query.all()`.

```
35 @app.route('/api/books', methods=['GET'])
36 def get_books():
37     books = Book.query.all()
38
39     result = []
40
41     for book in books:
42         result.append({
43             "id": book.id,
44             "title": book.title,
45             "author": book.author
46         })
47
48     return jsonify(result)
49
```

- To UPDATE (PUT): Find the book first: `book = Book.query.get(id)`. Update its attributes and then `db.session.commit()`.

```
50 @app.route('/api/books/<int:id>', methods=['PUT'])
51 def update_book(id):
52     book = Book.query.get(id)
53
54     if not book:
55         return jsonify({"message": "Book not found"}), 404
56
57     data = request.get_json()
58
59     book.title = data['title']
60     book.author = data['author']
61
62     db.session.commit()
63
64     return jsonify({
65         "message": "Book updated successfully"
66     })
67
```

- To DELETE (DELETE): Find the book, then use `db.session.delete(book)` followed by `db.session.commit()`.

```

68 @app.route('/api/books/<int:id>', methods=['DELETE'])
69 def delete_book(id):
70     book = Book.query.get(id)
71
72     if not book:
73         return jsonify({"message": "Book not found"}), 404
74
75     db.session.delete(book)
76     db.session.commit()
77
78     return jsonify({
79         "message": "Book deleted successfully"
80     })
81
82 if __name__ == '__main__':
83     app.run(port=5000)

```

Testing with curl

1. Start the Server: `python book_api.py`

```

PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pedriangco_Setubal_Siastres_lab_7\flask-db-lab> python book_api.py
* Serving Flask app 'book_api'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit

```

- a. Add a Book: Bash curl -X POST

```

http://127.0.0.1:5000/api/books \
-H "Content-Type: application/json" \
-d '{"title": "The Great Gatsby", "author": "F. Scott Fitzgerald"}'

```

```

PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pedriangco_Setubal_Siastres_lab_7\flask-db-lab> Invoke-WebRequest `
>> -Uri "http://127.0.0.1:5000/api/books" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{"title": "The Great Gatsby", "author": "F. Scott Fitzgerald"}'

message
-----
Book added successfully

```

2. Retrieve Books: Bash curl <http://127.0.0.1:5000/api/books>

```

PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pedriangco_Setubal_Siastres_lab_7\flask-db-lab> Invoke-WebRequest `
>> -Uri "http://127.0.0.1:5000/api/books" `
>> -Method GET

author          id title
-----
F. Scott Fitzgerald 1 The Great Gatsby

```